

Coding Standards

1. Consistency

- Maintain a consistent coding style across the project for readability and maintainability.
- Use consistent naming conventions, indentation, and formatting throughout the codebase.

2. Naming Conventions

2.1 HTML

- Use lowercase for HTML tags and attribute names.
- Use descriptive and meaningful class names.
- For IDs, use camelCase (e.g., userProfile) and avoid using numbers at the start.

Example:

```
<div class="user-profile">  
  
  <h2>User Profile</h2>  
  
</div>
```

2.2 CSS

- Use lowercase letters and hyphens for class and ID names (e.g., .main-header, .footer-section)
- Avoid using underscores in class or ID names.
- Group related styles together.

2.3 PHP

- Use snake_case for variables and functions (e.g., \$user_name, get_user_data()).
- Use UPPERCASE_SNAKE_CASE for constants (e.g., MAX_USERS).
- Use PascalCase for class names (e.g., UserProfile).
- Avoid single-letter variable names. Use descriptive names instead (e.g., \$user_id instead of \$u).

Example:

```
define('MAX_USERS', 100); // Constant
```

```
$user_name = "John"; // Variable
```

3. Comments and Documentation

- HTML: Use comments to explain complex sections of markup
- CSS: Comment sections of stylesheets for easier navigation (e.g., `/* Header styles */`).
- PHP: Use docstrings for functions and classes to explain the purpose, arguments, and return values.

4. Formatting and Indentation

- Use 2 spaces for indentation in HTML and CSS
- Use 4 spaces for indentation in PHP.
- Limit lines to 80 characters for readability.

Example:

```
/* Good CSS indentation */
```

```
.header {
```

```
    background-color: #333;
```

```
    padding: 10px;
```

```
}
```

```
footer {
```

```
    background-color: #f1f1f1;
```



```
    padding: 20px;
```

```
}
```

5. Error Handling

- Use try-catch blocks in PHP to handle errors gracefully.

Example:

```
try {  
  
    $result = calculate_area(5, 10);  
  
} catch (Exception $e) {  
  
    echo "Error occurred: " . $e->getMessage();  
  
}
```

6. Import Formatting

- In PHP, group imports as follows:
 1. Standard library imports
 2. Third-party library imports
 3. Local imports

Example

```
# Standard library imports  
include_once 'functions.php';  
  
# Third-party library imports  
require 'vendor/autoload.php';  
  
# Local imports  
require_once 'config.php';
```

7. URL Formatting (For Web Applications)

- Use lowercase letters and separate words with hyphens in URLs (e.g., <https://example.com/user-profile>).
- Avoid underscores in URLs.

8. Template Style

- Use semantic HTML in templates to make the code easily understandable.
- Ensure proper indentation and consistency for clarity

Example

```
<html>

<head>

  <title>My Website</title>

</head>

<body>

  <header>

    <h1>Welcome to my website!</h1>

  </header>

  <main>

    <section>

      <h2>About Us</h2>

      <p>Some description about the website...</p>

    </section>

  </main>

</body>


```

```
</html>
```

9. Code Readability

- Break down complex logic into smaller, reusable functions.
- Avoid cryptic variable names and use meaningful names that describe the purpose of the variable.

Example

```
function get_user_data($user_id) {

    // Fetch user data from database

    return $user_data;

}
```

10. Code Reusability

- Encapsulate reusable logic into functions or classes to avoid code duplication.

Example:

```
// Function to check user authentication

function check_authentication($user_id) {

    // Code to check if user is authenticated

}
```

11. Testing and Quality Assurance

- Use testing frameworks like PHPUnit for testing PHP functions and methods. Write unit tests for critical functionality.

Example:

```
use PHPUnit\Framework\TestCase;
```

```
class UserTest extends TestCase {

    public function test_get_user_data() {

        $user_id = 1;

        $result = get_user_data($user_id);

        $this->assertEquals('John Doe', $result['name']);

    }

}
```

12. Security

- Sanitize user inputs to prevent SQL injection and other security vulnerabilities.
- Use prepared statements for database queries.

Example:

```
// Using prepared statements in PHP
```

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE email = ?");
```

```
$stmt->execute([$email]);
```